

Diagnosing and Resolving PCIe BAR and I/O-Memory Mapping Failures for an AMD Radeon AI PRO R9700 eGPU on macOS with TinyGPU and tinygrad

Executive summary

The most important conclusion from the supplied session is that the observed failure should **not be treated as a single generic “PCIe BAR mapping bug.”** The evidence separates at least three distinct failure classes:

1. **Host/DriverKit PCI resource failure:** macOS enumerates the PCI function but TinyGPU cannot obtain or access a valid BAR aperture. Typical symptoms are zero-sized/unassigned BARs, failed `GetBARInfo`, Memory Space Enable being clear, or all-ones reads from every register.
2. **GPU initialization / indirect-VRAM-access failure after successful BAR mapping:** BAR0/BAR2/BAR5 exist and BAR5 register reads work, but the GPU-internal path used to access VRAM outside the small BAR0 aperture does not work. **This is what the supplied session experienced after one power cycle.**
3. **GPU VM/queue-programming failure after both BAR mapping and firmware discovery work:** BARs, discovery, VRAM, SDMA and queue setup can all succeed, yet compute fails because the programmed GPU virtual address or queue descriptor is wrong. **This is where the supplied session ultimately ended: the live blocker had moved to compute-MQD/HQD address encoding, not macOS BAR mapping.**

That distinction materially changes the debugging strategy.

The current tinygrad AMD initialization path immediately maps **BAR0 as VRAM, BAR2 as 64-bit doorbells, and BAR5 as 32-bit MMIO**, then executes `_run_discovery()`. The discovery routine reads `mmRCC_CONFIG_MEMSIZE` at register `0xde3`; if BAR0 covers all reported VRAM it reads the IP-discovery structure through BAR0, otherwise it accesses VRAM indirectly through the RSMU/MM_INDEX-style registers. The public TinyGPU side of tinygrad does not directly mmap PCI BARs in Python: `RemoteMMIOInterface` converts accesses into `MMIO_READ` / `MMIO_WRITE` RPCs, while `MAP_BAR` obtains BAR metadata from TinyGPU.app.

The supplied session recorded an R9700 at **PCI 1002:7551, gfx1201**, with **BAR0 = 256 MiB, BAR2 = 2 MiB, BAR5 = 512 KiB**, and GPU-reported VRAM size `34,208,743,424` bytes. Because 256 MiB is far smaller than the approximately 32 GiB of VRAM, `large_bar=False`; discovery therefore depends on the indirect BAR5/RSMU path. In the bad state, `RCC_CONFIG_MEMSIZE` still returned `0x7f70`, and the three BARs mapped at their expected sizes, but indirect VRAM reads returned a repeating `[0x00000000, 0xffffffff, 0xffffffff, 0xffffffff]` pattern at both the discovery-table location and VRAM offset

zero. A TinyGPU reset/FLR, a thirty-second wait, and BAR0 resize did not repair it. After a true cold cycle with the Thunderbolt/USB4 path disconnected, the extension again reported ready and IP discovery subsequently succeeded. (User-supplied session transcript, JSONL lines 1679, 1751, 1760–1764 and 1587.)

That observation is consistent with independent R9700 work: the `mac-amdgpu` DriverKit project reports a working `1002:7551` R9700 over Thunderbolt with **BAR5 register MMIO plus only the low 256 MiB of VRAM exposed by BAR0**, and obtains IP discovery through the GPU's indirect VRAM-access mechanism.

¹ Linux likewise treats BAR0 as the visible VRAM aperture and BAR2 as the doorbell aperture rather than assuming BAR0 must span all VRAM. The upstream AMDGPU doorbell code specifically obtains its doorbell base and length from PCI resource 2 and rejects an unset resource. ² Therefore **a 256 MiB BAR0 on this topology is not, by itself, evidence of a mapping failure and should not be made the primary remediation target.**

The R9700 itself is an RDNA 4, 32-GB workstation GPU with a PCIe 5.0 x16 host interface; AMD's current product documentation confirms those characteristics. ³ Both current R9700-on-macOS community implementations and upstream Linux source associate the device with PCI ID `1002:7551`; the working macOS DriverKit project additionally reports revision C0 and gfx1201. ¹

A second major architectural correction is necessary: **current TinyGPU is a DriverKit driver extension, or dext, not a traditional kernel extension.** TinyGPU's official documentation describes installation as a macOS driver extension and currently lists macOS 13.0+ as the requirement. ⁴ Apple explicitly distinguishes DriverKit dexts, which run in user space, from legacy kexts, which execute inside the kernel. ⁵ Consequently, SIP, Auxiliary Kernel Collection rebuilding, Reduced Security, and traditional kext signing are **not normal runtime prerequisites for the released TinyGPU dext.** They become relevant when developing a locally signed dext under Apple's development workflow, or only if a separate legacy kext is actually involved. Apple requires Reduced Security and an AuxKC rebuild for third-party kexts on Apple silicon, whereas DriverKit uses the System Extensions installation/signing/entitlement mechanism. ⁶

The highest-priority engineering changes are therefore:

- instrument TinyGPU's PCI command register and every `GetBARInfo` result, including the crucial **BAR-number → DriverKit memory-index translation**;
- provide an RPC-level distinction between "BAR exists," "BAR can be synchronously read through `IOPCIDevice::MemoryRead*`," and "GPU register returned all ones";
- classify `large_bar=False` as normal and separately validate the BAR5 indirect-VRAM path;
- instrument DMA/IOMMU state only after register MMIO has been proven, because an IOMMU failure affects DMA addressing and does not normally explain a synchronous CPU-side BAR5 register read failing;
- retain a minimal read-only transport probe so model loading and full AMD firmware initialization are not necessary to reproduce BAR failures;
- treat the session's **present compute-MQD/HQD address problem as a separate downstream bug**, rather than continuing to change PCI BAR allocation.

The end-to-end diagnostic model should be:

Attribute	Observed session value	Interpretation
<code>large_bar</code>	<code>false</code>	BAR0 cannot directly address discovery structure near end of VRAM
healthy discovery	<code>gc 12.0.1</code> , <code>mmhub 4.1.0</code> , <code>nbif 6.3.1</code> , <code>sdma 7.0.1</code>	Device discovery eventually succeeds
bad indirect VRAM state	repeating <code>0, ffffffff</code> , <code>ffffffff, ffffffff</code>	Not equivalent to “BAR0 failed to map”
TinyGPU post-cold-cycle status	<code>Extension is ready!</code>	Dext activation succeeds

These values appear repeatedly in the supplied transcript, notably JSONL lines 1679, 1751, 1760–1764 and 1587. They are compatible with the independently developed `mac-amdgpu` R9700 driver, which reports PCI `1002:7551`, BAR5 register access, a 256-MiB BAR0 aperture, and successful indirect IP-discovery access on real R9700 hardware. ¹

The nominal card specifications are also coherent with the observation: AMD specifies the R9700 as RDNA 4 with 32 GB GDDR6 and PCIe 5.0 x16. ⁸ Thunderbolt obviously does not expose a native x16 PCIe 5.0 host link to the Mac; the enclosure and Thunderbolt/USB4 fabric tunnel PCIe transactions and become part of the resource-allocation topology. Consequently, the exact enclosure, bridge controller, Thunderbolt generation, firmware revision, and host port are highly relevant variables even though the transcript does not definitively specify all of them.

What tinygrad actually does before inference

The current upstream `AMDev` constructor performs:

```
self.vram      = self.pci_dev.map_bar(0)
self.doorbell64 = self.pci_dev.map_bar(2, fmt='Q')
self.mmio      = self.pci_dev.map_bar(5, fmt='I')

self._run_discovery()
```

It therefore cannot reach model inference if BAR access or IP discovery fails.

Discovery then does, in substance:

```
mmRCC_CONFIG_MEMSIZE = 0xde3
self.vram_size = self.rreg(mmRCC_CONFIG_MEMSIZE) << 20
self.large_bar = self.vram.nbytes >= self.vram_size

tmr_offset = self.vram_size - (64 << 10)
```

```

disc_tbl = (
    self.vram.view(tmr_offset, 10 << 10)[: ]
    if self.large_bar
    else self._read_vram(tmr_offset, 10 << 10)
)

```

and rejects the result if the AMD binary/IP-discovery signatures are wrong.

The indirect `_read_vram()` path used when BAR0 is small writes GPU registers `0x06` and `0x00` and reads `0x01`; that means a correctly mapped BAR5 can coexist with a failed discovery operation when the GPU-internal indirect VRAM mechanism is not responding properly. (User-supplied transcript, JSONL line 1679.) This is precisely why `discovery signatures mismatch` is not sufficient evidence to diagnose a host BAR mapping failure.

The public TinyGPU client side reinforces the distinction. `RemoteCmd` contains separate operations for `MAP_BAR`, `MMIO_READ`, `MMIO_WRITE`, configuration reads/writes, system-memory mapping and reset, and `RemoteMMIOInterface` sends individual reads and writes over that RPC protocol. The most useful TinyGPU enhancement is therefore to return and log each intermediate PCI/DriverKit result rather than collapsing failures into a later `0xffffffff` value.

TinyGPU's official quick-start ultimately recommends an AMD workload such as `DEV=AMD python3 -m tinygrad.llm`.⁴ But for this class of failure that is too much machinery: a three-element Tensor realization, or preferably a transport-only PCI/MMIO probe, exercises the same device construction without introducing model loading, graph compilation, kernel compilation or inference state. A public R9700 TinyGPU issue uses exactly that sort of tiny Tensor operation and reaches the same discovery assertion.⁹

Why the session's original failure was downstream of BAR mapping

The strongest session observation is this combination:

- BAR0, BAR2 and BAR5 reported plausible nonzero sizes.
- `RCC_CONFIG_MEMSIZE` over register MMIO returned the correct-looking `0x7f70`.
- The indirect VRAM accessor returned systematic bad data.
- Reset/FLR did not repair it.
- A true power removal followed by reconnect restored discovery.
- After restoration, the native runner reported `arch_discovery_status: discovered_from_ip_table`, valid GFX/MMHUB/NBIF/SDMA versions and all three BAR sizes. (User-supplied transcript, JSONL lines 1751, 1760–1764 and 1587.)

A host-level failure to map BAR5 cannot easily explain a correct BAR5 `RCC_CONFIG_MEMSIZE` read while only the GPU's indirect VRAM operation returns garbage. The better diagnosis is therefore **“BAR/MMIO transport alive, GPU-internal indirect VRAM path not operational in that power state.”** The session attributed this to incomplete PSP/ROM initialization; that is a reasonable working hypothesis, especially because the cold cycle repaired it, but the transcript did not capture a definitive hardware PSP status register before discovery, so “PSP did not boot” should remain a hypothesis rather than a proven causal fact.

Linux's initialization sequence is instructive here. Modern AMDGPU initialization explicitly waits for the PSP bootloader on newer graphics IP before ASIC initialization, and its source has distinct direct-MMIO, indirect VRAM, firmware, reset, GART and queue phases. ¹⁰ That architecture supports treating indirect-VRAM readiness as a separate layer above PCI BAR mapping.

After the cold cycle, the session progressed substantially further: doorbells and queue activity were observed and the remaining failure was an RS64 exception interpreted by the session as a page fault/misaligned-address problem associated with MQD/HQD address-field encoding. That later failure is categorically **not** evidence that macOS failed to map a PCI BAR. It belongs in the GPU VM/queue layer and should be fixed independently. (User-supplied session transcript, final reconstructed findings around JSONL lines 1860–1872.)

Failure mechanisms and root-cause matrix

DriverKit BAR semantics that matter most

Apple's current PCIDriverKit interface makes one subtle detail especially important:

`IOPCIDevice::GetBARInfo()` takes a **BAR index** but returns a distinct **memory index**, along with BAR size and BAR type:

```
kern_return_t GetBARInfo(
    uint8_t barIndex,
    uint8_t *memoryIndex,
    uint64_t *barSize,
    uint8_t *barType
);
```

¹¹

The `memoryIndex` is what the synchronous `MemoryRead8/16/32/64` and `MemoryWrite*` APIs expect. `MemoryRead32`, for example, takes `memoryIndex` plus an offset; Apple documents that an error encountered by the read causes the destination value to be set to `-1`, which for `uint32_t` appears as `0xffffffff`. ¹²

This creates a high-value code-review target in TinyGPU:

Never assume `barIndex == memoryIndex`.

For a device with 64-bit BARs, BAR register numbering has gaps because a 64-bit BAR occupies two PCI configuration-space BAR slots, while DriverKit's memory-range indexing need not mirror those register numbers. Any implementation that receives TinyGPU's logical BAR5 and calls `MemoryRead32(5, ...)` without first translating BAR5 through `GetBARInfo(5, &memoryIndex, ...)` is suspect. The supplied session ultimately demonstrates that the installed build can access the R9700's relevant apertures, so this is not the best explanation for the recovered machine, but it is a first-class failure mode to instrument in the general TinyGPU path. Apple's API shape itself is the reason to test it. ¹³

Apple also explicitly requires a PCI endpoint driver to enable Memory Space Enable and Bus Master Enable when configuring the endpoint; if the driver unloads or crashes, macOS disables them. ¹⁴ A dext that reconnects after a crash but fails to restore the PCI command bits can therefore enumerate and read configuration space while MMIO does not function.

For legacy IOKit/kext code, the analogous APIs are different.

`IOPCIDevice::getDeviceMemoryWithRegister()` returns an `IODeviceMemory` for a BAR, and `mapDeviceMemoryWithRegister()` returns an `IOMemoryMap` or `nullptr`; `IODeviceMemory` is an `IOMemoryDescriptor` describing a device physical-memory range. ¹⁵ `IOService::mapDeviceMemoryWithIndex()` likewise returns an `IOMemoryMap`. ¹⁶ These are **legacy kernel-side concepts**, not evidence that current TinyGPU necessarily calls those methods.

Similarly, the relevant public modern type is `IOMemoryMap`, not a public class called `IOMappedMemory`. `IOMemoryDescriptor::CreateMapping()` produces an `IOMemoryMap` and returns `kIOReturnSuccess` or an error. ¹⁷

Root-cause comparison

Possible root cause	Expected diagnostic indicators	Fit to supplied session	Remediation / decisive test
TinyGPU dext not activated or not attached	<code>systemextensionsctl</code> does not show activated/enabled; no <code>IOUserServer/service</code> ; TinyGPU cannot probe GPU	Low after cold cycle: TinyGPU reported <code>Extension is ready!</code>	Verify extension state and IORegistry match before touching GPU registers. Reinstall only if activation signature path fails. ¹⁸
DriverKit PCI entitlement/matching failure	Dext exists but never matches <code>1002:7551</code> ; activation errors; entitlement absent from code signature	Low for current session , but common during custom dext development	Inspect signed entitlements and <code>IOPCIMatch</code> ; <code>com.apple.developer.driverkit.transport</code> . Apple validates entitlements during system-extension activation. ¹⁹
BAR index used instead of DriverKit <code>memoryIndex</code>	<code>GetBARInfo</code> succeeds but <code>MemoryRead*</code> using raw BAR number returns <code>0xffffffff</code> or wrong aperture	Worth auditing; unlikely current recovered blocker	Log <code>barIndex</code> , <code>memoryIndex</code> , size, type for every BAR; only use returned <code>memoryIndex</code> in <code>MemoryRead*</code> / <code>MemoryWrite*</code> . ¹³

Possible root cause	Expected diagnostic indicators	Fit to supplied session	Remediation / decisive test
Memory Space Enable clear	PCI COMMAND lacks MEM bit; config space works; aperture reads fail/all ones	Possible after a dext crash/reload; not supported by recovered logs	Read COMMAND before and after <code>Open</code> ; set <code>kiOPCICommandMemorySpace</code> , verify readback. <code>Open</code> is explicitly responsible for this. ²⁰
Bus Master Enable clear	Register MMIO may work but DMA/SDMA/system-memory transfers fail	Not a discovery explanation; relevant later	Enable BME and read it back before DMA setup. L equivalent is <code>setBusMasterEnable(true)</code> . ²¹
BAR resource absent/unassigned or upstream bridge window inadequate	<code>GetBARInfo</code> fails/size=0; config BAR base invalid; Linux analogue reports <code>IORESOURCE_UNSET</code>	Does not fit the successful session BAR sizes	Capture all six config BAR registers and <code>GetBARInfo</code> compare bridge windows. Linux AMDGPU refuses unset BAR2 doorbell resource. ²²
256-MiB BAR0 incorrectly treated as fatal	BAR0 is 256 MiB although card has ~32 GB VRAM; <code>large_bar=False</code>	Definitely present, but not a bug by itself	Keep indirect VRAM path. Do not make ReBAR a prerequisite. A separate R9700 DriverKit implementation succeeds with 256-MiB BAR0. ¹
GPU indirect VRAM/RSMU mechanism unready after power event	BAR5 normal register reads work; BAR sizes valid; indirect VRAM returns patterned zeros/all ones; cold power removal changes behavior	High; best fit to historical session failure	Test BAR5 direct MMIO and RSMU separately. If one RSMU fails, stop blaming host mapping. Full cold removal was effective in this session.
PCIe power-state/link/tunnel problem	Config function remains visible but endpoint MMIO intermittently returns all ones; behavior tied to hot-plug/cold boot	Plausible contributor to historical state	Log PCI power status/link state, repeat across warm reset, enclosure power cycle and full cable disconnect. Use a second port/enclosure when possible.
IOMMU/DART DMA mapping bug	MMIO registers work, but GPU DMA to system RAM faults, returns corrupted data or addresses outside device aperture	Low for discovery; important for later SDMA/compute	Instrument <code>IODMACommand</code> preparation and generation of GPU-visible bus addresses. Do not diagnose an all BAR5 read as an IOMMU issue. <code>IODMACommand</code> specifically to convert memory references to device addresses. ²³

Possible root cause	Expected diagnostic indicators	Fit to supplied session	Remediation / decisive test
Incorrect <code>IOMemoryDescriptor</code> lifecycle	DMA preparation/wiring errors such as not-ready/no-resources; stale/dangling shared-memory mapping	Relevant only to system-memory/DMA portions	Pair prepare/complete operations and retain descriptors through the transfer. Legacy <code>prepare()</code> on non-pageable memory and must be paired with <code>complete()</code> . ²⁴
Client shared-memory mapping failure	Dext works internally but <code>IOConnectMapMemory</code> fails or client receives wrong descriptor	Possible only if TinyGPU exposes mapped shared buffers that way	Instrument <code>IOUserClient::CopyClientMemoryForType</code> to ensure same descriptor for the same type and log errors
Cache/order/alignment error	Correct values via <code>DriverKit MemoryRead*</code> , wrong or stale values through a custom mapped path; 64-bit doorbells especially sensitive	Possible later; weak fit for discovery	Use synchronous <code>MemoryRead*</code> as an oracle, add memory barriers and width/alignment assertions to compare 32/64-bit accesses.
PSP/firmware/GPU initialization failure	Host MMIO alive but GPU-managed data structures/indirect paths unavailable; cold boot changes state	High historical plausibility but not conclusively proven	Add pre-discovery hardware-state probes and distinguish them from BAR failures. Linux initialization explicitly separates PSP/ASIC initialization from PCI resource setup. ¹⁰
MQD/HQD/GPU-VA address encoding error	BAR/discovery/SDMA work; doorbell hit occurs; compute engine reports page/misalignment fault	High for the session's final blocker	Fix MQD address-field splitting and add host-only serialization contract tests. Do not resize BARs or change <code>DriverKit</code> mapping in response.

A June 2026 public TinyGPU R9700 issue describes an M4/Tahoe 26.5.1 + ACASIS setup in which config space was readable but MMIO appeared as all ones and attributed that result to unassigned BARs.⁹ That report is useful evidence of the symptom class, but its root-cause conclusion should not be generalized to the supplied session: the session explicitly obtained nonzero BAR sizes and a correct `RCC_CONFIG_MEMSIZE` MMIO value, then later achieved full IP discovery. Another public R9700 issue shows yet another failure class: the dext matched but its `IOUserServer` did not launch.²⁶ These reports reinforce the need for layered error classification instead of mapping every `discovery signatures mismatch` to BAR allocation.

IOMMU, VT-d and Apple platform security

For an Apple-silicon TinyGPU host, **VT-d is the wrong architectural label**; VT-d is Intel terminology. The relevant operational question is whether DriverKit's DMA mapping path gives the GPU valid I/O bus addresses. Apple's `IODMACommand` abstraction exists to convert memory references into I/O bus addresses and explicitly supersedes `IOMemoryCursor`. ²⁷

This distinction gives a useful diagnostic rule:

A broken IOMMU/DMA mapping can make GPU DMA fail even while BAR5 register reads work. It should not normally be the first explanation when a CPU-issued `IOPCIDevice::MemoryRead32` of BAR5 itself returns `0xffffffff`.

For legacy kernel drivers, `IOMemoryCursor` is therefore not the preferred modern mechanism; Apple documents `IODMACommand` as its replacement. ²³ If TinyGPU's system-memory RPC path is synthesizing or exposing physical addresses without the appropriate DMA mapping abstraction, that deserves a separate audit, especially once SDMA begins moving data between the R9700 and Mac memory.

The Apple T2 is likewise a conditional concern, not a generic TinyGPU requirement. The R9700/TinyGPU community reports under discussion are Apple-silicon hosts, for which the relevant boot-security model differs from T2-era Intel Macs. Legacy kext installation on Apple silicon requires Reduced Security and explicit kernel-extension permission; Apple explains that such kexts are incorporated into an Auxiliary Kernel Collection. ²⁸ DriverKit dexts instead run in user space and are installed through System Extensions. ²⁹

SIP should therefore be handled carefully. Apple's own DriverKit sample recommends temporarily disabling SIP and enabling `systemextensionsctl developer` for certain locally signed development configurations. ³⁰ That does **not** imply that an Apple-approved production TinyGPU installation should require SIP disabled. For released TinyGPU, first verify actual signature and entitlements rather than changing platform security.

TinyGPU instrumentation and robustness patches

Instrument the DriverKit boundary first

The most valuable log point is the TinyGPU handler for `MAP_BAR`, before tinygrad sees the result.

At driver start or device-open time, log:

- `IOPCIDevice::Open(...)` result;
- BDF from `GetBusDeviceFunction`;
- vendor/device/revision/class configuration values;
- PCI COMMAND before and after enabling memory/bus-master decoding;
- each of BAR0 through BAR5 via `GetBARInfo`;
- returned `memoryIndex`, `barSize`, and `barType`;
- a safe read probe from each aperture through `IOPCIDevice::MemoryRead*`;

- link speed where available using `GetLinkSpeed`;
- reset result and post-reset PCI COMMAND/BAR state.

`GetBARInfo` and `GetLinkSpeed` return `kern_return_t`, so successful operation should be `kIOReturnSuccess`; every non-success value should be logged, not converted into a zero size.³¹ `IOMemoryDescriptor::CreateMapping` likewise explicitly returns `kIOReturnSuccess` on success and another DriverKit error on failure.³² Useful error symbols to preserve instead of collapsing them include `kIOReturnBadArgument`, `kIOReturnNoMemory`, `kIOReturnNoResources`, `kIOReturnNotAligned`, `kIOReturnNotAttached`, `kIOReturnNotOpen`, `kIOReturnNotPermitted`, `kIOReturnNotReady`, `kIOReturnNotResponding`, `kIOReturnTimeout`, `kIOReturnUnsupported` and `kIOReturnVMError`; these are defined in Apple's DriverKit error-code set. The exact failure code is provider/operation dependent, so TinyGPU should record it rather than predict it.

The important exception is the synchronous aperture read/write API: `MemoryRead32` itself returns `void`; Apple says it writes `-1` to the output on read error.¹² Thus an all-ones value must be interpreted alongside a successful `GetBARInfo`, the memory index, PCI COMMAND state and comparison reads—not by itself.

A diagnostic log should look like this:

```
TinyGPU[pci] open kr=0x00000000 bdf=04:00.0 vid=1002 did=7551 rev=c0

TinyGPU[pci] command.before=0x0000
TinyGPU[pci] command.after =0x0006 mem_space=1 bus_master=1

TinyGPU[bar] bar=0 get_info.kr=0x00000000
TinyGPU[bar] bar=0 memory_index=0 size=0x10000000 type=...
TinyGPU[bar] bar=0 probe.offset=0x0 read32=0x..... valid=1

TinyGPU[bar] bar=2 get_info.kr=0x00000000
TinyGPU[bar] bar=2 memory_index=1 size=0x00200000 type=...

TinyGPU[bar] bar=5 get_info.kr=0x00000000
TinyGPU[bar] bar=5 memory_index=2 size=0x00080000 type=...
TinyGPU[bar] bar=5 probe.reg=0x0de3 value=0x00007f70

TinyGPU[map] logical_bar=5 -> memory_index=2
```

The example `memory_index` values above are deliberately illustrative; **log the actual returned indices and never hard-code those numbers.**

Apple documents `os_log` as the unified logging mechanism, and DriverKit also exposes logging support including `IOLog` / `IOLogv`.³³ In a development build, a dedicated subsystem such as `org.tinygrad.tinygpu` and categories `pci`, `bar`, `dma`, `power`, and `rpc` would make filtering substantially easier.

Pseudo-patch for the TinyGPU BAR handler

The TinyGPU DriverKit implementation is not present in the public tinygrad Python source examined here, so the following is deliberately a source-level **pseudo-patch**, not a claim about the current binary's exact class names.

```
struct BARState {
    bool valid = false;
    uint8_t bar = 0xff;
    uint8_t memoryIndex = 0xff;
    uint8_t type = 0;
    uint64_t size = 0;
};

kern_return_t PCIIface::refreshBAR(uint8_t bar, BARState& out) {
    out = {};
    out.bar = bar;

    uint8_t memIndex = 0xff;
    uint8_t barType = 0xff;
    uint64_t size = 0;

    kern_return_t kr =
        pciDevice->GetBARInfo(bar, &memIndex, &size, &barType);

    os_log_with_type(log, OS_LOG_TYPE_INFO,
        "BAR%u GetBARInfo kr=0x%x mem=%u size=0x%llx type=0x%x",
        bar, kr, memIndex, size, barType);

    if (kr != kIOReturnSuccess)
        return kr;

    if (size == 0) {
        os_log_with_type(log, OS_LOG_TYPE_ERROR,
            "BAR%u has zero-sized resource", bar);
        return kIOReturnNoResources;
    }

    out.valid = true;
    out.memoryIndex = memIndex;
    out.size = size;
    out.type = barType;
    return kIOReturnSuccess;
}

kern_return_t PCIIface::readBAR32(
    uint8_t bar, uint64_t offset, uint32_t& value) {
```

```

BARState s;
kern_return_t kr = refreshBAR(bar, s);
if (kr != kIOReturnSuccess) return kr;

if (offset > s.size || s.size - offset < sizeof(uint32_t))
    return kIOReturnBadArgument;

value = 0xfeedface;
pciDevice->MemoryRead32(s.memoryIndex, offset, &value);

os_log_with_type(log,
    value == UINT32_MAX ? OS_LOG_TYPE_ERROR : OS_LOG_TYPE_DEBUG,
    "BAR%u mem=%u off=0x%llx read32=0x%08x",
    bar, s.memoryIndex, offset, value);

return kIOReturnSuccess;
}

```

The robustness point is not the exact logging syntax; it is the preservation of four different facts:

```

logical BAR number
    ≠
DriverKit memoryIndex
    ≠
BAR size/type
    ≠
actual value returned by the device

```

`GetBARInfo` exists specifically to provide that metadata. ³⁴

At `Start()` /open, add explicit PCI decode-state verification:

```

uint16_t command = 0;
pciDevice->ConfigurationRead16(/* PCI COMMAND offset */, &command);

const uint16_t wanted =
    command |
    kIOPCICommandMemorySpace |
    kIOPCICommandBusMaster;

if (wanted != command)
    pciDevice->ConfigurationWrite16(/* PCI COMMAND offset */, wanted);

uint16_t verify = 0;
pciDevice->ConfigurationRead16(/* PCI COMMAND offset */, &verify);

```

```
os_log_with_type(log, OS_LOG_TYPE_INFO,
    "PCI_COMMAND before=0x%04x after=0x%04x MSE=%d BME=%d",
    command, verify,
    !(verify & kIOPCICommandMemorySpace),
    !(verify & kIOPCICommandBusMaster));
```

Apple explicitly assigns this responsibility to endpoint drivers. ²⁰

After `Reset()`, repeat that entire sequence. A successful FLR followed by an MMIO failure is otherwise ambiguous because the system may have disabled or altered decode state around reset/unload.

Add a synchronous DriverKit read as a mapping oracle

If TinyGPU currently creates its own `IOMemoryDescriptor` / `IOMemoryMap` around the PCI aperture, add a debug mode in which every suspect mapped read is compared against `IOPCIDevice::MemoryRead32`:

```
mapped BAR5[0xde3] = 0xffffffff
MemoryRead32(memIndex, 0xde3*4) = 0x00007f70
```

would immediately prove that the **custom mapping layer is broken while DriverKit's PCI aperture access works**.

Conversely:

```
mapped BAR5[0xde3] = 0xffffffff
MemoryRead32(memIndex, 0xde3*4) = 0xffffffff
GetBARInfo(BAR5) = success, size=0x80000
PCI_COMMAND.MSE = 1
```

moves the investigation toward endpoint power/link/device state instead of `IOMemoryMap`.

For a legacy kext implementation, the equivalent diagnostics would inspect `getDeviceMemoryWithRegister()` and `mapDeviceMemoryWithRegister()`; the former returns an `IODeviceMemory`, while the latter returns `IOMemoryMap*` or null. ³⁵ A null mapping is therefore an actionable result and must not be dereferenced or silently converted to an all-ones device value.

Improve tinygrad's failure classification

The current `_run_discovery()` eventually throws the generic:

```
AssertionError: discovery signatures mismatch
```

after either a direct BAR0 access **or a completely different indirect BAR5/RSMU operation**. That obscures the most important debugging boundary.

A more diagnostic implementation would be:

```
class AMDev:
    def __init__(self, pci_dev, reset_mode=False):
        self.pci_dev, self.devfmt = pci_dev, pci_dev.pcibus
-       self.vram, self.doorbell64, self.mmio = (
-           self.pci_dev.map_bar(0),
-           self.pci_dev.map_bar(2, fmt='Q'),
-           self.pci_dev.map_bar(5, fmt='I'))
+       self._bar_info = {
+           b: self.pci_dev.bar_info(b) for b in (0, 2, 5)
+       }
+       self.vram = self.pci_dev.map_bar(0)
+       self.doorbell64 = self.pci_dev.map_bar(2, fmt='Q')
+       self.mmio = self.pci_dev.map_bar(5, fmt='I')
+
+       if AM_DEBUG:
+           print("AM BAR topology:",
+               {b: self._bar_info[b] for b in (0,2,5)})

        self._run_discovery()

    def _run_discovery(self):
        mmRCC_CONFIG_MEMSIZE = 0xde3
-       self.vram_size = self.rreg(mmRCC_CONFIG_MEMSIZE) << 20
+       memsize = self.rreg(mmRCC_CONFIG_MEMSIZE)
+       if memsize in (0, 0xffffffff):
+           raise RuntimeError(
+               f"BAR5 MMIO transport invalid: "
+               f"RCC_CONFIG_MEMSIZE={memsize:#010x}")
+
+       self.vram_size = memsize << 20
+       self.large_bar = self.vram.nbytes >= self.vram_size

        tmr_offset = self.vram_size - (64 << 10)
-       disc_tbl = self.vram.view(...)[: ] if self.large_bar \
-           else self._read_vram(...)
+       if self.large_bar:
+           source = "BAR0-direct"
+           disc_tbl = self.vram.view(tmr_offset, 10 << 10)[: ]
+       else:
+           source = "BAR5-indirect-RSMU"
+           disc_tbl = self._read_vram(tmr_offset, 10 << 10)
+
+ 
```

```

+     first_words = memoryview(disc_tbl).cast('I')[:16].tolist()
+     if AM_DEBUG:
+         print(f"AM discovery source={source} "
+               f"vram={self.vram_size:#x} "
+               f"bar0={self.vram.nbytes:#x} "
+               f"words={[hex(x) for x in first_words]}")
+
+     if not self._valid_discovery_signatures(disc_tbl):
+         if not self.large_bar:
+             raise RuntimeError(
+                 "AMD discovery invalid over indirect VRAM path; "
+                 "BAR5 register MMIO may be alive while RSMU/VRAM "
+                 "access is not ready")
+         raise RuntimeError(
+             "AMD discovery invalid through direct BAR0 aperture")

```

That change alone would have prevented much of the ambiguity in the supplied session.

A second classification should recognize a repeated all-ones/zero pattern and explicitly advise that the failure is **downstream of BAR5 mapping if a known BAR5 register remains readable**.

Instrument DMA separately

DriverKit memory allocation and PCI BAR access should not be conflated. `IOMemoryDescriptor` describes memory, while `IOMemoryMap` represents a mapping into an address space. `IOMemoryDescriptor::CreateMapping()` creates such a map. ¹⁷ `IODMACommand`, separately, converts memory into addresses usable for device DMA and supersedes `IOMemoryCursor`. ²³

For every TinyGPU system-memory allocation, therefore log at least:

```

allocation id
requested bytes/alignment
descriptor length
CreateMapping kr
IOMemoryMap::GetAddress()
IOMemoryMap::GetLength()
IODMACommand preparation kr
number of DMA segments
each GPU-visible bus address + length
direction
completion status

```

`IOMemoryMap::GetAddress()` and `GetLength()` are the public accessors for the mapped virtual range. ³⁶

If a legacy descriptor path is present, remember that `prepare()` and `complete()` must be paired for pageable memory; `prepare()` wires the memory for I/O. ²⁴ A leaked or prematurely completed DMA mapping can cause GPU page faults while all BAR MMIO remains healthy.

Prioritized reproduction and isolation plan

The priority order should deliberately progress from **macOS resource assignment** → **DriverKit aperture access** → **GPU indirect VRAM access** → **DMA** → **GPU VM/queue** → **inference**. Jumping directly into tinygrad LLM inference makes the failure surface much larger than necessary.

Establish an immutable baseline

Before each hardware experiment, save:

```
sw_vers
uname -a

systemextensionsctl list

csrutil status

system_profiler SPThunderboltDataType
system_profiler SPPCIDDataType

ioreg -p IOService -l -w0 > /tmp/ioreg-full.txt

/Applications/TinyGPU.app/Contents/MacOS/TinyGPU status
```

Do not infer the macOS version from a related GitHub issue. The supplied session itself does not conclusively identify it. Current TinyGPU documentation specifies macOS 13.0+; public R9700 reports happen to include Tahoe 26.5.1, while `mac-amdgpu` targets Tahoe 26.x, but those are separate environments. ³⁷

Inspect the actual installed driver extension rather than assuming bundle metadata:

```
find /Applications/TinyGPU.app/Contents/Library/SystemExtensions \
    -maxdepth 2 -print

codesign -dv --verbose=4 /Applications/TinyGPU.app 2>&1

codesign -d --entitlements :- /Applications/TinyGPU.app 2>/dev/null
```

Then run `codesign -d --entitlements :-` on the discovered `.dext` path.

For a PCI DriverKit dext, verify the signed entitlement set includes the appropriate DriverKit/PCI transport authorization; Apple's `com.apple.developer.driverkit.transport.pci` entitlement controls PCI matching. ³⁸ The containing application needs the System Extension capability to activate the driver. ¹⁹

Do not change SIP or Secure Boot during this first baseline. A functioning released TinyGPU install should be tested under its intended security configuration before development-mode workarounds are introduced.

Capture TinyGPU and DriverKit logs

Use unified logging while attaching the enclosure and launching the minimal probe:

```
log stream \  
  --style compact \  
  --level debug \  
  --predicate '  
    process == "TinyGPU" OR  
    eventMessage CONTAINS[c] "tinygpu" OR  
    eventMessage CONTAINS[c] "driverkit" OR  
    eventMessage CONTAINS[c] "IOPCIDevice" OR  
    eventMessage CONTAINS[c] "1002" OR  
    eventMessage CONTAINS[c] "7551"  
  '
```

A debug TinyGPU build with its own subsystem should instead be filtered directly:

```
log stream \  
  --style compact \  
  --level debug \  
  --predicate 'subsystem == "org.tinygrad.tinygpu"'
```

DriverKit is user-space, so unified logging is the primary instrumentation path. Apple documents both unified `os_log` and DriverKit-compatible logging facilities. ³⁹

DTrace should be secondary. SIP prevents DTrace from inspecting protected system processes, even as root, although Apple notes that developers can still inspect their own applications. ⁴⁰ Thus use DTrace only on a development TinyGPU/IUserServer configuration for which tracing is permitted; do not disable SIP merely to turn DTrace into the first diagnostic tool.

Run a transport-only Python repro

Before constructing `AMDev`, test exactly what the TinyGPU transport reports:

```
from tinygrad.runtime.support.system import APLRemotePCIDevice
```

```
pci = APLRemotePCIDevice("AMD", "usb4")

for bar in (0, 2, 5):
    try:
        print(f"BAR{bar}: {pci.bar_info(bar)}")
    except Exception as exc:
        print(f"BAR{bar}: ERROR {exc!r}")

bar5 = pci.map_bar(5, fmt="I")

# Fixed pre-discovery register used by tinygrad.
rcc = bar5[0xDE3]
print(f"RCC_CONFIG_MEMSIZE = 0x{rcc:08x}")
print(f"VRAM bytes          = 0x{rcc << 20:x}")
```

The session's healthy transport should be approximately:

```
BAR0 size = 0x10000000
BAR2 size = 0x00200000
BAR5 size = 0x00080000
RCC_CONFIG_MEMSIZE = 0x00007f70
```

The exact base-address portion returned by `bar_info()` is topology-dependent; compare sizes and stability, not hard-coded physical bases. The public tinygrad client obtains BAR information through the `MAP_BAR` RPC.

This produces four immediately useful classifications:

```
BAR5 absent/size zero
  -> host/DriverKit PCI resource problem

BAR5 valid, RCC == 0xffffffff
  -> BAR aperture/device decode/power problem

BAR5 valid, RCC == 0x7f70, direct registers stable
  -> host BAR5 mapping is working

RCC valid but discovery later fails
  -> investigate direct BAR0 or indirect VRAM path,
      not generic BAR mapping
```

Reproduce the indirect-VRAM operation without full AMD initialization

For the session's 256-MiB BAR0, this is the critical isolation test.

A minimal implementation mirroring tinygrad's pre-discovery accessor is conceptually:

```
from tinygrad.runtime.support.system import APLRemotePCIDevice

pci = APLRemotePCIDevice("AMD", "usb4")
mmio = pci.map_bar(5, fmt="I")

memsize_mib = mmio[0xDE3]
vram_size = memsize_mib << 20

def read_vram_u32(addr: int) -> int:
    assert addr % 4 == 0

    # Same pre-discovery indirect path used by AMDev.
    mmio[0x06] = addr >> 31
    mmio[0x00] = (addr & 0x7fffffff) | 0x80000000
    return mmio[0x01]

for base in (
    0,
    vram_size - (64 << 10),
):
    words = [
        read_vram_u32(base + i * 4)
        for i in range(16)
    ]
    print(hex(base), [f"0x{x:08x}" for x in words])
```

This test is far more discriminating than `Tensor(...).to("AMD")`.

The key decision table is:

```
RCC correct + indirect VRAM correct
-> run full AMDev discovery

RCC correct + indirect VRAM repeating 0/ffffffff
-> BAR5 is demonstrably mapped;
    GPU-internal indirect VRAM path is failing

RCC itself ffffffff
-> investigate BAR5/PCI decode/link first
```

Compare warm and cold state transitions

For the exact historical session failure, run the read-only probe after each transition and record results in a matrix:

Transition	BAR info	RCC register	indirect VRAM offset 0	discovery table
clean boot				
TinyGPU server restart				
dext reload				
TinyGPU <code>reset()</code> / FLR				
enclosure power off/on with cable attached				
enclosure off + cable physically disconnected				
host shutdown + full enclosure cold cycle				

The supplied transcript already provides one strong A/B result: software reset did not change the broken RSMU data, whereas a later full cold cycle restored discovery. That result deserves to be reproduced several times before encoding “cold cable disconnect required” as a permanent workaround.

If enclosure details are available, add:

```
manufacturer/model
Thunderbolt generation
bridge ASIC
bridge firmware
host Mac model
host port
cable model/length
PCIe link width/rate as reported by macOS
```

Public TinyGPU reports demonstrate that enclosure/bridge behavior can materially affect enumeration and driver attachment. [41](#)

Only then test DMA and IOMMU state

Once BAR5 and discovery are proven, allocate the smallest possible system buffer and log every DMA segment generated for it.

The invariants should be:

```
CPU writes known pattern
  ↓
descriptor/map valid
  ↓
IODMACommand prepared
  ↓
GPU-visible DMA address recorded
  ↓
GPU page table points to that address
  ↓
SDMA copies to/from VRAM
  ↓
CPU verifies exact bytes
```

`IODMACommand` is the appropriate abstraction for translating memory references to I/O bus addresses and may involve mapping hardware; that is precisely where an IOMMU problem should appear. ²³

Use a progression such as:

```
4 KiB system -> VRAM -> system
64 KiB
2 MiB
non-contiguous multi-page allocation
repeated allocation/free
```

Do not begin with a model weight buffer.

Gate compute separately from transport

After transport, discovery and SDMA all pass:

1. install one GPU page-table mapping;
2. read the PTE back;
3. flush GC and MMHUB TLBs;
4. run SDMA through the mapped GPU VA;
5. create one compute ring;
6. serialize one MQD;
7. verify the MQD bytes before activation;
8. activate HQD;
9. read HQD register fields;
10. ring one doorbell;
11. execute the smallest possible kernel;
12. verify a single output word.

This is already close to the supplied session's native C0 gate, and that session successfully progressed far enough to expose a separate MQD/HQD address problem. That sequence should remain the compute acceptance test.

For the current MQD work, add no-hardware serialization assertions for each hardware-defined address field. The supplied session noted, for example, that `CP_HQD_PQ_BASE` represents address bits `[39:8]`, while the readback appeared inconsistent with the intended `0x2000007000` ring address. Fix that encoding and prove the exact MQD bytes before changing any macOS BAR behavior.

Run actual tinygrad inference last

Only after the preceding gates are green should the test advance to:

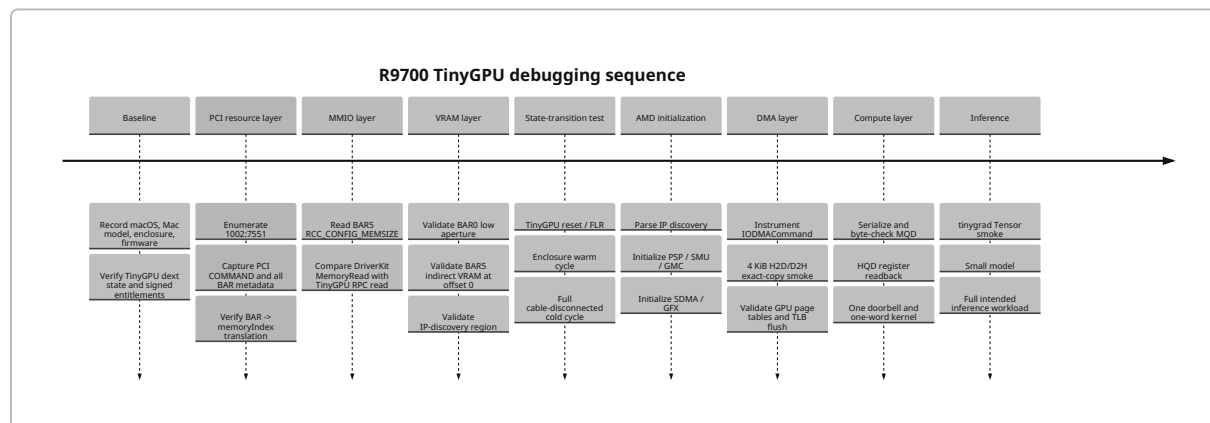
```
DEV=AMD python3 - <<'PY'
from tinygrad import Tensor
x = Tensor([1, 2, 3], device="AMD")
print((x + 1).numpy())
PY
```

and then:

```
DEV=AMD python3 -m tinygrad.llm
```

The latter is the current official TinyGPU usage path. ⁴

The debugging timeline should therefore be:



Source-level cross-checks and community evidence

Apple references and what they imply

The most important Apple sources for this investigation are:

PCIDriverKit `IOPCIDevice`. Apple defines it as the provider object for custom PCI hardware and explicitly states that the endpoint driver must enable Memory Space Enable and Bus Master Enable whenever it configures the PCI device. ¹⁴ This is the primary reference for TinyGPU's PCI-side code.

`IOPCIDevice::GetBARInfo`. This is the authoritative API for translating a BAR number into the DriverKit memory index, size and type. ¹¹ It should be the central logging point in `MAP_BAR`.

`IOPCIDevice::MemoryRead*` / `MemoryWrite*`. These provide a reference path for synchronous aperture access. In particular, `MemoryRead32` reports read failure through an all-ones output value rather than a return code. ⁴² This explains why an isolated `0xffffffff` requires supporting metadata before it can be called an unassigned BAR.

`IOMemoryDescriptor` and `IOMemoryMap`. DriverKit's `IOMemoryDescriptor` describes memory, and `CreateMapping()` creates an `IOMemoryMap`. ¹⁷ These APIs are relevant to shared/system memory but should not be casually conflated with PCI BAR metadata.

`IOUserClient::CopyClientMemoryForType`. If TinyGPU exposes shared mappings through a user client, `IOConnectMapMemory` results in this method being called to obtain the descriptor. Apple specifies that a given memory type for a given user client should return the same descriptor. ²⁵ Instrument it if the DEXT-side access works but the app-side mapping does not.

`IODMACommand`. Apple explicitly states that it supersedes `IOMemoryCursor` and converts memory references into I/O bus addresses. ²⁷ This should be the center of any DART/IOMMU investigation rather than BAR0/BAR5 mapping.

DriverKit entitlements and System Extensions. DriverKit drivers run in user space; their activation checks signatures and entitlements. Apple documents the PCI transport entitlement and the System Extension installation requirements. ⁴³

For a legacy-kext comparison, Apple's older IOKit API exposes `IODeviceMemory`, `getDeviceMemoryWithRegister()`, `mapDeviceMemoryWithRegister()`, `setMemoryEnable()` and `setBusMasterEnable()`. ⁴⁴ Those APIs are useful conceptual references, but they should not be described as TinyGPU's current implementation unless its dext source specifically shows them.

Linux AMDGPU behavior

Linux is especially useful as a source-level behavioral reference because it separates the relevant hardware layers.

For AMD doorbells, upstream `amdgpu_doorbell_init()` checks **PCI resource 2**, rejects it if `IORESOURCE_UNSET`, and obtains both base and size through the PCI resource API. ²² This agrees with tinygrad mapping BAR2 as the R9700 doorbell aperture.

AMDGPU also derives the CPU-visible VRAM aperture from PCI BAR0 rather than assuming the aperture equals installed VRAM. ⁴⁵ That is the critical comparison for the R9700's 256-MiB BAR0.

The upstream driver has explicit indirect VRAM access logic using MM_INDEX/MM_DATA-style registers and rejects unaligned position/size. ¹⁰ Tinygrad's small-BAR fallback is therefore conceptually aligned with an established AMD driver technique: limited CPU-visible VRAM does not mean the rest of VRAM is unusable.

Modern AMDGPU initialization also explicitly waits for PSP bootloader readiness on newer graphics generations before subsequent ASIC initialization. ¹⁰ This supports the session's decision to look beyond PCI mapping when indirect GPU-managed state was unavailable, although it does not by itself prove that the session's bad post-power-cycle state was specifically a PSP boot failure.

The upstream Linux tree contains `0x7551` in AMD GPU support code, and independent macOS R9700 bring-up identifies the same device as gfx1201. ¹

R9700-specific community implementations

The strongest independent community reference is `lemonade-sdk/mac-amdgpu`. Its current README reports full R9700 bring-up over Thunderbolt 5 using a native PCIDriverKit dext. It specifically reports:

- PCI `1002:7551`, revision C0;
- BAR5 for register MMIO;
- BAR0 as the low 256 MiB of VRAM;
- IP discovery through an indirect VRAM mechanism;
- successful PSP, SMU, GMC, CP/MES and SDMA initialization. ¹

That makes the project valuable as a **behavioral control** for TinyGPU. In particular, its success argues strongly against the proposition that an R9700 must have a 32-GB BAR0 to initialize correctly on macOS.

It should not, however, be copied blindly. Its README describes a development configuration requiring its own DriverKit entitlements and SIP-disabled setup. ¹ Those requirements belong to that project's development/signing arrangement and are not automatically TinyGPU requirements.

A separate experimental gfx1201 project likewise targets the R9700/RX 9070 XT device family through a macOS DriverKit dext and direct userspace queue programming. ⁴⁶ It is particularly relevant once the problem advances beyond BAR mapping into MES/MQD/HQD behavior.

Public TinyGPU issue #16714 is worth following because it is almost exactly the nominal scenario in this research request: R9700 `0x7551`, Apple-silicon Mac, Tahoe 26.5.1 and Thunderbolt eGPU with all-ones discovery reads. ⁹ Its diagnosis should nevertheless be verified against actual `GetBARInfo`, PCI COMMAND and DriverKit `memoryIndex` logs before treating “macOS never assigns eGPU BARs” as established fact. The supplied session itself provides a counterexample in which the same class of card had valid nonzero BAR apertures and eventually booted successfully.

Issue #16726 is a useful reminder that a dext can be “activated enabled” and even match the PCI object while the IOUserServer still fails to launch. ²⁶ That belongs above BAR mapping in the decision tree.

Older OpenCore/eGPU forum material is much less directly applicable. Apple's historical eGPU support documentation and traditional eGPU community registries concern Apple's graphics-driver path on Intel

Macs, not TinyGPU's compute-only DriverKit path on Apple silicon. ⁴⁷ Use those threads for enclosure/Thunderbolt topology knowledge, not as authority on current TinyGPU PCI resource handling.

Recommendations, patches, and next steps

Immediate priorities for this exact session

First, freeze the known-good PCI state. Do not change BAR sizing, SIP, ReBAR settings or the TinyGPU installation while working on the current compute fault. The supplied session has already demonstrated:

```
1002:7551 enumeration      PASS
TinyGPU dext ready         PASS
BAR0/BAR2/BAR5 present    PASS
BAR5 RCC register access   PASS
IP discovery after cold boot PASS
SDMA-related operations    progressed successfully
compute doorbell observed  PASS
```

The active failure therefore belongs after PCI BAR mapping.

Second, fix and contract-test the MQD/HQD address encoding. The session's strongest downstream evidence points to incorrectly reconstructed compute addresses and an RS64 page/misalignment fault. Add pure host-side tests that serialize each MQD address field from a known 64-bit GPU VA and assert the exact dwords/bytes before running hardware again. Only after those pass should the C0 one-word kernel test be rerun.

Third, independently patch TinyGPU diagnostics anyway. The historical RSMU failure is likely to recur, and today TinyGPU/tinygrad provides insufficient information to tell these states apart:

```
BAR unmapped
BAR valid but MSE disabled
BAR valid but endpoint unresponsive
BAR5 direct MMIO valid but indirect VRAM invalid
discovery table genuinely malformed
```

Those should become separate errors.

Recommended TinyGPU RPC extension

A versioned diagnostic command would eliminate guesswork:

```
struct TinyGPUBarInfoV1 {
    uint32_t version;
    uint32_t kr;
```

```

uint8_t  bar;
uint8_t  memory_index;
uint8_t  bar_type;
uint8_t  reserved;

uint64_t size;

uint16_t pci_command;
uint16_t pci_status;

uint32_t probe_value;
uint32_t flags;
};

```

Expose:

```

GET_BAR_INFO
GET_PCI_STATE
MMIO_READ_CHECKED
GET_DMA_MAP_INFO
GET_LINK_STATE

```

and make `MMIO_READ_CHECKED` return both a transport status and the device value. That prevents `0xffffffff` from carrying two meanings—"the device actually returned all ones" and "DriverKit could not perform the access."

Recommended BAR mapping policy

For R9700 specifically:

```

BAR0  -> accept 256 MiB
        treat as low VRAM aperture
        large_bar=False is valid

BAR2  -> require nonzero doorbell aperture
        assert 8-byte alignment for 64-bit doorbells

BAR5  -> require sufficient register aperture
        verify RCC_CONFIG_MEMSIZE before AMD initialization

VRAM beyond BAR0
    -> use validated indirect VRAM path

```

```
ReBAR -> optimization / topology experiment,  
        not required for correctness
```

This policy agrees with both current tinygrad's logic and a working independent R9700 macOS implementation. ¹

A blind `resize_bar(0)` should not be part of the normal recovery ladder. Resizing an endpoint BAR also requires compatible upstream address-window assignment, and a larger aperture is unnecessary if the indirect VRAM path is healthy. The supplied session's `resize_bar(0)` did not change its broken post-power-cycle state, while a later cold cycle restored discovery.

Recommended failure taxonomy in user-facing errors

Replace generic assertions with structured messages such as:

```
TINYGPU_PCI_NOT_ATTACHED  
TINYGPU_PCI_BAR_MISSING(bar=5)  
TINYGPU_PCI_MEMORY_DECODE_DISABLED  
TINYGPU_PCI_MMIO_ALL_ONES(bar=5, offset=...)  
AM_RCC_MEMSIZE_INVALID  
AM_BAR0_SMALL_USING_INDIRECT_VRAM  
AM_INDIRECT_VRAM_UNRESPONSIVE  
AM_DISCOVERY_SIGNATURE_INVALID  
AM_DMA_MAPPING_FAILED  
AM_GPUVM_PAGE_FAULT  
AM_COMPUTE_QUEUE_ADDRESS_INVALID
```

That taxonomy maps directly onto different remediation actions instead of inviting repeated kext/SIP/ReBAR changes.

Recommended security workflow

For the released TinyGPU package:

```
verify signature  
verify entitlements  
verify system extension activated  
leave SIP/security policy at supported defaults
```

DriverKit runs in user space and system-extension activation already validates the driver's signature and entitlements. ⁴⁸

For a locally modified TinyGPU dext:

```
use an isolated development machine/volume
follow Apple's DriverKit development-signing workflow
use systemextensionsctl developer mode when required
change SIP only when Apple's development procedure actually requires it
restore normal security after testing
```

Apple's sample explicitly documents the special development workflow. ⁴⁹

For a true legacy kext fork, security requirements change substantially: on Apple silicon, third-party kexts require Reduced Security, user authorization and AuxKC integration. ²⁸ Kernel-cache signing discussions should therefore be omitted from a normal TinyGPU-deft diagnosis unless a legacy kext is actually present.

High-value comparative experiments

The best remaining research would compare exactly the same R9700 across:

Experiment	What it answers
TinyGPU vs <code>mac-amdgpu</code> on same Mac/enclosure	Is the failure TinyGPU-specific or topology/device-state-specific?
Different Thunderbolt port	Host controller/path sensitivity
Different certified TB cable	Signal/tunnel stability
Different enclosure	Bridge firmware/resource-window dependency
Same enclosure after warm reset vs cable-disconnected cold boot	Persistent auxiliary-power/device-state hypothesis
Same card in Linux with upstream AMDGPU	Capture authoritative PCI BAR capabilities, ReBAR capability, PSP boot and normal resource layout
<code>MemoryRead32</code> vs TinyGPU's ordinary read implementation	DriverKit aperture good but custom mapping bad
BAR0 256 MiB with indirect VRAM vs attempted ReBAR	Proves whether resizing has any correctness benefit
DMA with 4-KiB buffer before compute queue	Separates host IOMMU from GPU queue errors

A Linux capture from the same physical card would be particularly valuable:

```
lspci -nnvv -s <BDF>
lspci -xxxx -s <BDF>
```

```
cat /sys/bus/pci/devices/<BDF>/resource  
dmesg | grep -iE 'amdgpu|BAR|VRAM|doorbell|PSP|IOMMU'
```

The comparison should focus on BAR capability sizes versus assigned aperture sizes, PCI COMMAND, Resizable BAR capability, power-state transitions, BAR2 doorbells, and the AMDGPU initialization sequence.

Open questions and limitations

The supplied transcript does **not** definitively establish the Mac model, exact macOS build, exact enclosure/Thunderbolt controller, cable topology or bridge firmware. Those should therefore remain metadata fields in the next diagnostic capture rather than being inferred from the very similar public M4/Tahoe/ACASIS reports.

The public tinygrad source exposes the Python-side TinyGPU RPC and current `AMDev` BAR/discovery path, but the source of the installed TinyGPU DriverKit binary itself was not available in the reviewed public tinygrad code. Consequently, references above to TinyGPU's internal `GetBARInfo`, `MemoryRead*`, `IOMemoryDescriptor` or `IOMemoryMap` implementation are **recommended instrumentation/pseudo-patches**, not assertions that the production dext presently implements BAR access in that exact way.

Likewise, the session's cold-cycle result strongly localizes its historical failure below BAR5 MMIO and inside GPU initialization/indirect VRAM access, but it does not provide a definitive hardware register proving that PSP firmware itself was the component that failed to boot. The defensible conclusion is narrower: **the GPU-internal indirect VRAM/discovery path was unavailable despite functioning PCI/BAR5 register access, and complete power removal restored it.**

The current session state is still more decisive: after that recovery, the R9700 reached successful IP discovery and progressed into queue/doorbell execution. The next engineering action should therefore be the **MQD/HQD address-encoding repair and C0 compute contract test**, while the BAR/DriverKit changes above should be implemented as diagnostics and resilience improvements for the next recurrence rather than as the presumed fix for the present compute failure.

1 mac-amdgpu/README.md at main · lemonade-sdk/mac-amdgpu · GitHub

https://github.com/lemonade-sdk/mac-amdgpu/blob/main/README.md?utm_source=chatgpt.com

2 22 linux/drivers/gpu/drm/amd/amdgpu/amdgpu_doorbell_mgr.c at master · torvalds/linux · GitHub

https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/amdgpu_doorbell_mgr.c?utm_source=chatgpt.com

3 AMD Introduces New Radeon Graphics Cards and Ryzen Threadripper Processors at COMPUTEX 2025 :: Advanced Micro Devices, Inc. (AMD)

https://ir.amd.com/news-events/press-releases/detail/1253/amd-introduces-new-radeon-graphics-cards-and-ryzen-threadripper-processors-at-computex-2025?utm_source=chatgpt.com

4 37 egpu for mac - tinygrad docs

https://docs.tinygrad.org/tinygpu/?utm_source=chatgpt.com

5 29 43 48 DriverKit security for macOS - Apple Support

https://support.apple.com/en-mide/guide/security/seca48c92d43/web?utm_source=chatgpt.com

- 6 28 **Change security settings on the startup disk of a Mac with Apple silicon - Apple Support**
[https://support.apple.com/guide/mac-help/change-security-settings-startup-disk-a-mac-mchl768f7291/mac?](https://support.apple.com/guide/mac-help/change-security-settings-startup-disk-a-mac-mchl768f7291/mac?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://support.apple.com/guide/mac-help/change-security-settings-startup-disk-a-mac-mchl768f7291/mac?utm_source=chatgpt.com)
- 7 14 20 **IOPCIDevice | Apple Developer Documentation**
https://developer.apple.com/documentation/pcidriverkit/iopcidevice?utm_source=chatgpt.com
- 8 **AMD Radeon™ AI PRO R9700**
[https://www.amd.com/pt/products/graphics/workstations/radeon-ai-pro/ai-9000-series/amd-radeon-ai-pro-r9700.html?](https://www.amd.com/pt/products/graphics/workstations/radeon-ai-pro/ai-9000-series/amd-radeon-ai-pro-r9700.html?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://www.amd.com/pt/products/graphics/workstations/radeon-ai-pro/ai-9000-series/amd-radeon-ai-pro-r9700.html?utm_source=chatgpt.com)
- 9 **macOS eGPU BAR Mapping: MMIO returns 0xFFFFFFFF — AMD R9700 on Apple Silicon M4 · Issue #16714 · tinygrad/tinygrad · GitHub**
https://github.com/tinygrad/tinygrad/issues/16714?utm_source=chatgpt.com
- 10 **linux/drivers/gpu/drm/amd/amdgpu/amdgpu_device.c at master · torvalds/linux · GitHub**
https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/amdgpu_device.c?utm_source=chatgpt.com
- 11 31 34 **GetBARInfo | Apple Developer Documentation**
https://developer.apple.com/documentation/pcidriverkit/iopcidevice/getbarinfo?utm_source=chatgpt.com
- 12 13 42 **MemoryRead32 | Apple Developer Documentation**
https://developer.apple.com/documentation/pcidriverkit/iopcidevice/memoryread32-60hg9?utm_source=chatgpt.com
- 15 35 44 **mapDeviceMemoryWithRegister | Apple Developer Documentation**
[https://developer.apple.com/documentation/kernel/iopcidevice/1811470-mapdevicememorywithregister?](https://developer.apple.com/documentation/kernel/iopcidevice/1811470-mapdevicememorywithregister?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://developer.apple.com/documentation/kernel/iopcidevice/1811470-mapdevicememorywithregister?utm_source=chatgpt.com)
- 16 **mapDeviceMemoryWithIndex | Apple Developer Documentation**
https://developer.apple.com/documentation/kernel/iopcidevice/1810261-mapdevicememorywithindex?utm_source=chatgpt.com
- 17 **IOMemoryDescriptor | Apple Developer Documentation**
https://developer.apple.com/documentation/driverkit/iomemorydescriptor?utm_source=chatgpt.com
- 18 **Installing System Extensions and Drivers | Apple Developer Documentation**
[https://developer.apple.com/documentation/systemextensions/installing-system-extensions-and-drivers?](https://developer.apple.com/documentation/systemextensions/installing-system-extensions-and-drivers?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://developer.apple.com/documentation/systemextensions/installing-system-extensions-and-drivers?utm_source=chatgpt.com)
- 19 38 **Requesting Entitlements for DriverKit Development | Apple Developer Documentation**
[https://developer.apple.com/documentation/driverkit/requesting-entitlements-for-driverkit-development?](https://developer.apple.com/documentation/driverkit/requesting-entitlements-for-driverkit-development?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://developer.apple.com/documentation/driverkit/requesting-entitlements-for-driverkit-development?utm_source=chatgpt.com)
- 21 **setBusMasterEnable | Apple Developer Documentation**
https://developer.apple.com/documentation/kernel/iopcidevice/1811490-setbusmasterenable?utm_source=chatgpt.com
- 23 27 **IODMACommand | Apple Developer Documentation**
https://developer.apple.com/documentation/driverkit/iodmacommand?utm_source=chatgpt.com
- 24 **complete | Apple Developer Documentation**
https://developer.apple.com/documentation/kernel/iomemorydescriptor/1812740-complete?utm_source=chatgpt.com
- 25 **CopyClientMemoryForType | Apple Developer Documentation**
https://developer.apple.com/documentation/driverkit/iouserclient/copyclientmemoryfortype?utm_source=chatgpt.com
- 26 **R9700 (RDNA4) + Mac mini M4 + Acasis G4Pro: dext matches (IOEXTMatchCount=1) but IOUserServer never launches — BrokenPipeError · Issue #16726 · tinygrad/tinygrad · GitHub**
https://github.com/tinygrad/tinygrad/issues/16726?utm_source=chatgpt.com

30 49 Communicating between a DriverKit extension and a client app | Apple Developer Documentation

https://developer.apple.com/documentation/driverkit/communicating-between-a-driverkit-extension-and-a-client-app?utm_source=chatgpt.com

32 CreateMapping | Apple Developer Documentation

https://developer.apple.com/documentation/driverkit/iomemorydescriptor/createmapping?utm_source=chatgpt.com

33 Log | Apple Developer Documentation

https://developer.apple.com/documentation/driverkit/iodispatchqueue/log?utm_source=chatgpt.com

36 GetLength | Apple Developer Documentation

https://developer.apple.com/documentation/driverkit/iomemorymap/getlength?utm_source=chatgpt.com

39 IOLogv | Apple Developer Documentation

https://developer.apple.com/documentation/kernel/1575323-iologv?utm_source=chatgpt.com

40 Runtime Protections

https://developer.apple.com/library/archive/documentation/Security/Conceptual/System_Integrity_Protection_Guide/RuntimeProtections/RuntimeProtections.html?utm_source=chatgpt.com

41 M4/NVIDIA/AG02 on PDX vendor_id... · Issue #15760 · tinygrad/tinygrad · GitHub

https://github.com/tinygrad/tinygrad/issues/15760?utm_source=chatgpt.com

45 linux/drivers/gpu/drm/amd/amdgpu/gmc_v6_0.c at master · torvalds/linux · GitHub

https://github.com/torvalds/linux/blob/master/drivers/gpu/drm/amd/amdgpu/gmc_v6_0.c?utm_source=chatgpt.com

46 gfx1201 (RDNA4) lite:: GPU bring-up — macOS eGPU / Linux / Windows: build, run & ramp-up (kept updated) · GitHub

https://gist.github.com/powderluv/331ae9a351f786315670c3b3516c97b7?utm_source=chatgpt.com

47 Use an external graphics processor with your Mac – Apple Support (AU)

https://support.apple.com/en-au/102363?utm_source=chatgpt.com